

Unit 2. Introduction to ASP.NET

3 Hrs.

.NET and ASP.NET frameworks: .NET, .NET Core, Mono, ASP.NET Web Forms, ASP.NET MVC, ASP.NET Web API, ASP.NET Core, .NET Architecture and Design Principles, Compilation and Execution of .NET applications: CLI, MSIL and CLR, .NET Core in detail, .NET CLI: build, run, test and deploy .NET Core Applications

Unit 3. HTTP and ASP.NET Core

3 Hrs.

HTTP, Request and Response Message Format, Common web application architectures, MVC Pattern, ASP.NET Core Architecture Overview, Projects, and Conventions, ASP.NET and ASP.NET MVC

Yuba Raj Devkota
NET Centric (CSIT 6th Sem)
NCCS

.NET and ASP.NET Frameworks Overview

- **.NET Framework:** Windows-only development platform.
- **.NET Core:** Cross-platform, open-source version of .NET.
- **Mono:** Open-source implementation of .NET for Linux and mobile platforms.
- **ASP.NET Web Forms:** Event-driven web development using server controls.
- **ASP.NET MVC:** Web framework based on the Model-View-Controller architecture.
- **ASP.NET Web API:** Framework for building RESTful web services.
- **ASP.NET Core:** Modern cross-platform framework for web applications and APIs.

1. .NET Framework

.NET Framework is Microsoft's software development platform used to build Windows desktop applications, web applications, and services. It provides a large library (Framework Class Library - FCL) and a runtime called CLR (Common Language Runtime).

Features

- *Runs mainly on Windows.*
- *Supports languages like C#, VB.NET, and F#.*
- *Provides automatic memory management (Garbage Collection).*

```
using System;

class Program
{
    static void Main()
    {
        Console.WriteLine("Hello .NET Framework!");
    }
}
```

Output

Hello .NET Framework!

2. .NET Core

```
using System;  
  
Console.WriteLine("Hello .NET Core!");
```

Output

```
Hello .NET Core!
```

.NET Core is a modern, lightweight, open-source, and cross-platform version of .NET. It runs on Windows, Linux, and macOS.

Features

- *Cross-platform*
- *Faster performance*
- *Open source*
- *Ideal for cloud applications and microservices*

Example Use

- REST APIs
- Cloud applications
- Cross-platform software

3. Mono

Mono is an open-source implementation of Microsoft's .NET Framework that allows .NET applications to run on Linux, macOS, Android, and iOS.

Features

- *Cross-platform*
- *Supports mobile development*
- *Used before .NET Core became popular*

Example Use

- Running a C# application on Linux.

```
using System;

class Program
{
    static void Main()
    {
        Console.WriteLine("Running with Mono");
    }
}
```

Output

Running with Mono

4. ASP.NET Web Forms

ASP.NET Web Forms is Microsoft's event-driven framework for creating web applications. It works similarly to Windows Forms using drag-and-drop controls.

Features

- *Event-driven programming*
- *Server controls (Button, TextBox, GridView)*
- *Easy for beginners*

Output

When the button is clicked:

Welcome to ASP.NET Web Forms

Default.aspx

```
</> aspx

<asp:Button ID="btnHello"
    runat="server"
    Text="Click Me"
    OnClick="btnHello_Click" />

<asp:Label ID="lblMessage" runat="server"></asp:Label>
```

Default.aspx.cs

```
</> C#

protected void btnHello_Click(object sender, EventArgs e)
{
    lblMessage.Text = "Welcome to ASP.NET Web Forms";
}
```

5. ASP.NET MVC

ASP.NET MVC (Model-View-Controller) is a framework that separates an application into three parts:

- *Model → Data*
- *View → User Interface*
- *Controller → Business Logic*

This separation makes applications easier to maintain and test.

Controller

```
</> C#  
  
public class HomeController : Controller  
{  
    public IActionResult Index()  
    {  
        return Content("Welcome to ASP.NET MVC");  
    }  
}
```

Output

Welcome to ASP.NET MVC

6. ASP.NET Web API

ASP.NET Web API is a framework for building RESTful APIs that allow applications to exchange data using HTTP.

Features

- *Returns JSON or XML*
- *Used by web, mobile, and desktop applications*
- *Lightweight communication*

```
[ApiController]
[Route("api/hello")]
public class HelloController : ControllerBase
{
    [HttpGet]
    public string Get()
    {
        return "Hello from Web API";
    }
}
```

If we visit:

`https://localhost/api/hello`

Output

Hello from Web API

7. ASP.NET Core

ASP.NET Core is the latest, open-source, cross-platform framework for building web applications, REST APIs, and cloud-based applications. It combines the best features of MVC and Web API.

Features

- *Cross-platform*
- *High performance*
- *Open source*
- *Supports MVC, Razor Pages, Web API, and Blazor*
- *Runs on Windows, Linux, and macOS*

```
var builder = WebApplication.CreateBuilder(args);  
var app = builder.Build();  
  
app.MapGet("/", () => "Hello ASP.NET Core!");  
  
app.Run();
```

Open browser:

```
https://localhost:5001/
```

Output

```
Hello ASP.NET Core!
```

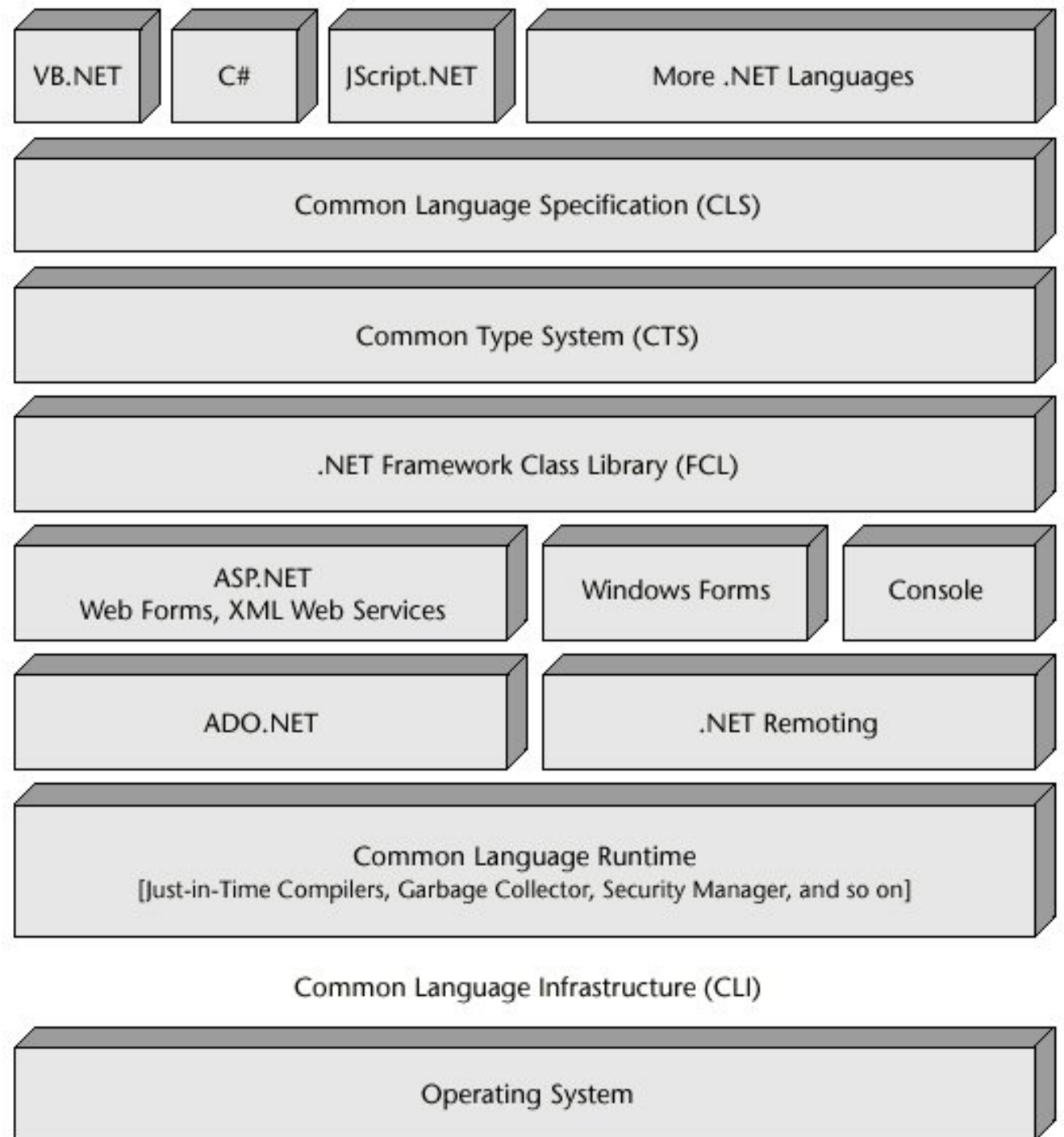
Technology	Platform	Main Purpose	Example Use
.NET Framework	Windows only	Desktop & Web applications	Windows Forms, WPF
.NET Core	Windows, Linux, macOS	Cross-platform applications	REST APIs, Cloud apps
Mono	Cross-platform	Run .NET apps on Linux/Mobile	Xamarin, Linux apps
ASP.NET Web Forms	Windows	Event-driven web applications	Company websites
ASP.NET MVC	Windows	Web applications using MVC pattern	E-commerce websites
ASP.NET Web API	Windows	RESTful APIs	Mobile app backend
ASP.NET Core	Cross-platform	Modern web apps & APIs	Cloud applications, Microservices

.NET Architecture and Design Principles

The .NET Architecture defines how a .NET application is organized and executed. It consists of languages (such as C#), the Common Language Runtime (CLR), the Framework Class Library (FCL), and the operating system.

Components

- **Programming Language:** C#, VB.NET, F#
- **Compiler:** Converts source code into MSIL.
- **CLR:** Executes the program.
- **FCL (Framework Class Library):** Provides built-in classes (String, List, File, etc.).
- **Operating System:** Windows, Linux, or macOS.



1. Programming Languages (Top Layer)

- **VB.NET, C#, JScript.NET, More .NET Languages:** These are the languages developers use to write code. The key feature here is that all these languages compile down to the same intermediate code, allowing them to work together seamlessly.

2. Language & Type Standards

- **Common Language Specification (CLS):** This is a set of rules that ensures language interoperability. It guarantees that code written in one language (e.g., C#) can use libraries written in another (e.g., VB.NET).
- **Common Type System (CTS):** This defines how data types are declared, used, and managed in the runtime. It ensures that an "integer" in C# is treated exactly the same as an "integer" in VB.NET.

3. The Library Layer

- **.NET Framework Class Library (FCL):** A massive collection of reusable classes, interfaces, and value types. It provides pre-written code for common tasks like file handling, string manipulation, and networking, so developers don't have to reinvent the wheel.

4. Application Models & Technologies

This layer represents the different types of applications and technologies you can build using the framework:

- **ASP.NET (Web Forms, XML Web Services):** Used for building dynamic websites, web applications, and web services.
- **Windows Forms:** Used for building traditional desktop applications with a Graphical User Interface (GUI).
- **Console:** Used for building command-line applications.
- **ADO.NET:** A set of components for data access. It allows applications to connect to data sources like SQL Server, Oracle, or XML files.
- **.NET Remoting:** A technology for communication between different application domains (allowing objects to interact across networks). *Note: This is an older technology largely replaced by WCF and Web APIs in modern .NET.*

5. The Execution Engine

- **Common Language Runtime (CLR):** This is the heart of the .NET Framework. It is the engine that actually runs your code.
 - **Just-in-Time (JIT) Compilers:** Converts the intermediate code into native machine code right before it runs.
 - **Garbage Collector:** Automatically manages memory by cleaning up objects that are no longer in use.
 - **Security Manager:** Enforces security policies.

6. Infrastructure & OS (Bottom Layer)

- **Common Language Infrastructure (CLI):** This is the international standard (ECMA/ISO) that describes the executable code and runtime environment. The CLR is Microsoft's implementation of the CLI.
- **Operating System:** The base layer (Windows, Linux, macOS) on which the entire framework runs. The CLI allows .NET to be portable across different operating systems.

Design Principles

1. Language Independence

Programs can be written in different .NET languages.

Example

- Module A → C#
- Module B → VB.NET
- Both can work together.

2. Platform Independence (.NET Core/.NET)

Run the same application on:

- Windows
- Linux
- macOS

3. Object-Oriented Programming

Supports:

- Encapsulation
- Inheritance
- Polymorphism
- Abstraction

4. Automatic Memory Management

CLR automatically removes unused objects using **Garbage Collection (GC)**.

```
</> C#  
  
Student s = new Student();
```

When `s` is no longer used, CLR automatically frees its memory.

5. Security

CLR provides:

- Type safety
- Exception handling
- Memory safety

.NET Architecture is the overall structure of the .NET platform consisting of programming languages, the CLR, Framework Class Library (FCL), and the operating system. It provides language independence, security, portability, and automatic memory management.

Compilation and Execution of .NET Applications

When we write a C# program, it is not executed directly. It goes through several stages.

(a) CLI (Common Language Infrastructure)

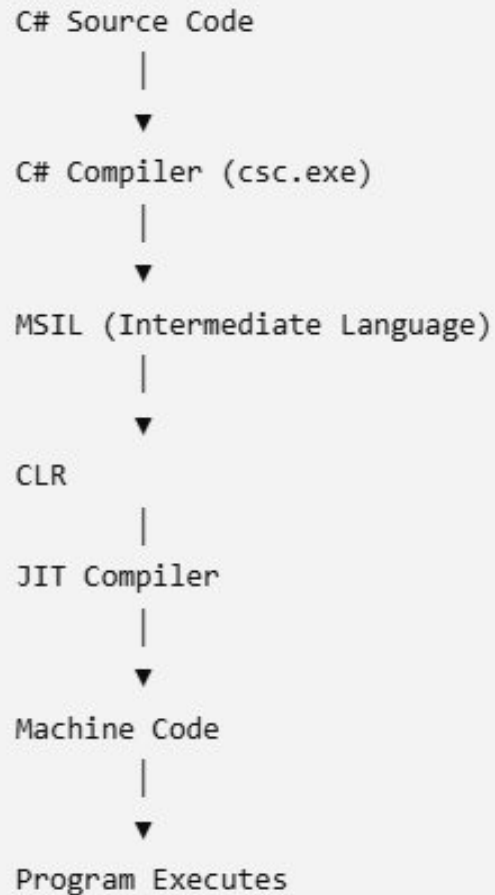
CLI is an international standard (ECMA/ISO) that defines how .NET languages and runtimes should work together. It specifies the rules for the runtime, intermediate language, metadata, and language interoperability.

Main Components of CLI

- **CTS (Common Type System)** – Defines common data types (int, string, bool, etc.).
- **CLS (Common Language Specification)** – Rules that all .NET languages follow to ensure interoperability.
- **MSIL (Microsoft Intermediate Language)** – Intermediate code generated by the compiler.
- **CLR (Common Language Runtime)** – Executes the program.

Example

A class written in VB.NET can be used by a C# application because both follow the CLI standard.



(b) MSIL (Microsoft Intermediate Language)

MSIL (also called IL/CIL) is the intermediate code generated by the C# compiler. It is CPU-independent and cannot run directly.

MSIL is converted to machine code only when the program runs.

Example

Source Code

```
</> C#  
  
Console.WriteLine("Hello");
```

After compilation

```
Hello.exe  
|  
Contains MSIL code
```

Responsibilities of CLR

- Executes programs
- JIT compilation
- Memory management
- Garbage collection
- Exception handling
- Security
- Thread management

(c) CLR (Common Language Runtime)

CLR is the execution engine of .NET. It converts MSIL into machine code using the Just-In-Time (JIT) compiler and provides services such as memory management, security, exception handling, and garbage collection.

Example

```
</> C#  
  
Console.WriteLine("Welcome");
```

MSIL



CLR



JIT Compiler



Machine Code



Output:

Welcome

JIT (Just-In-Time Compiler)

The JIT compiler converts MSIL into machine code only when the code is executed.

```
Console.WriteLine("Hello");
```

The first time this line runs:

```
MSIL
  ↓
JIT Compiler
  ↓
Machine Code
```

The generated machine code is then executed.

.NET Framework vs .NET Core

Feature	.NET Framework	.NET Core (.NET)
Platform	Windows only	Windows, Linux, macOS
Open Source	No	Yes
Performance	Good	Faster
Best For	Windows desktop applications	Web, cloud, APIs, microservices
Deployment	System-wide installation	Side-by-side versions supported

.NET CLI (Command Line Interface)

The .NET CLI (Command Line Interface) is a command-line tool used to create, build, run, test, publish, and deploy .NET applications without using Visual Studio. It is available on Windows, Linux, and macOS.

.NET CLI lets developers manage .NET projects using terminal commands.

1. `dotnet build` – Build the Application

The **build** command compiles the source code and checks for errors. It creates executable files (`.dll` or `.exe`) in the **bin** folder.

Syntax

```
</> Bash
```

```
dotnet build
```

Example

```
</> Bash
```

```
dotnet build MyApp.csproj
```

Output

```
Build succeeded.
```

```
0 Warning(s)
```

```
0 Error(s)
```

Use: Before running or deploying an application.

2. `dotnet run` – Run the Application

The **run** command builds (if necessary) and immediately executes the application.

Syntax

`</> Bash`

```
dotnet run
```

Example

Program.cs

`</> C#`

```
using System;
```

```
Console.WriteLine("Welcome to .NET Core!");
```

Command

`</> Bash`

```
dotnet run
```

Output

```
Welcome to .NET Core!
```

Use: Quickly test the application during development.

3. `dotnet test` – Test the Application

The **test** command runs unit tests written using frameworks like **xUnit**, **NUnit**, or **MSTest**.

Syntax

❏ Bash

```
dotnet test
```

```
using Xunit;

public class CalculatorTest
{
    [Fact]
    public void AddTest()
    {
        Assert.Equal(5, 2 + 3);
    }
}
```

Command

❏ Bash

```
dotnet test
```

Output

```
Passed! 1 test passed.
```

Use: Verify that the application works correctly before deployment.

4. `dotnet publish` – Deploy the Application

To deploy a .NET application, we use the **publish** command. It creates all files needed to run the application on another computer.

Syntax

</> Bash

```
dotnet publish
```

Example

</> Bash

```
dotnet publish -c Release
```

Output

```
Publishing completed.  
Files generated in:  
bin/Release/net8.0/publish/
```

The **publish** folder contains:

- Executable file (.exe)
- DLL files
- Configuration files
- Runtime dependencies

These files can be copied to a server or another computer for deployment.

Complete Example Workflow

Suppose you have a project named **HelloWorld**.

Step 1: Create a project

</> Bash

```
dotnet new console -n HelloWorld
```

Step 2: Move into the project

</> Bash

```
cd HelloWorld
```

Step 3: Build

</> Bash

```
dotnet build
```

Step 4: Run

</> Bash

```
dotnet run
```

Output:

```
Hello, World!
```

Step 5: Test (if test project exists)

</> Bash

```
dotnet test
```

Step 6: Publish for deployment

</> Bash

```
dotnet publish -c Release
```

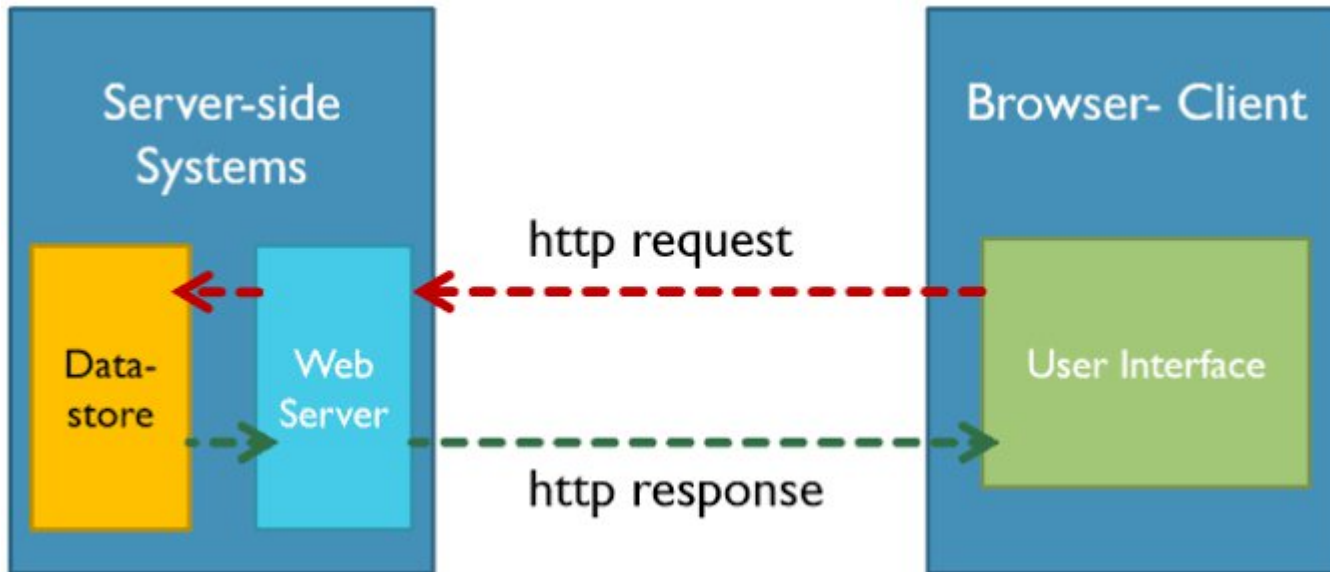
Published files will be available in:

```
bin/  
└─ Release/  
    └─ net8.0/  
        └─ publish/
```


HTTP (HyperText Transfer Protocol)

HTTP (HyperText Transfer Protocol) is the communication protocol used between a web browser (client) and a web server. It defines how requests are sent and how responses are received over the Internet.

- *Client: Sends an HTTP request (e.g., Browser, Mobile App)*
- *Server: Processes the request and sends an HTTP response (e.g., ASP.NET Core application)*



Example

When you type:

`https://www.google.com`

The browser sends an HTTP request:

`GET /`

Google's server returns an HTTP response containing the HTML page.

Stateless Nature of HTTP

HTTP is **stateless**, meaning the server does **not remember previous requests**.

Example:

Suppose you login to Facebook.

Request 1:
POST /login

Server verifies your credentials.

Next request:

GET /profile

The server does **not automatically remember** you logged in.

Therefore websites use:

- Cookies
- Sessions
- Tokens (JWT)

to maintain user login.

HTTP Methods (Verbs)

These define the action to perform.

Method	Purpose	Example
GET	Retrieve data	View student details
POST	Send/Create data	Add new student
PUT	Update entire resource	Update student information
PATCH	Update part of resource	Update student's phone number
DELETE	Delete data	Delete student

HTTP vs HTTPS

HTTP	HTTPS
Not secure	Secure
Data sent in plain text	Data encrypted
Port 80	Port 443
No SSL/TLS	Uses SSL/TLS certificate

Example

`http://college.com`

`https://college.com`

Always prefer **HTTPS** because it encrypts communication.

HTTP Request Message Format

Whenever a client sends data to the server, it follows a standard format.

Request Line

Headers

Blank Line

Body (optional)

Structure of HTTP Request

1. Request Line

Contains

- HTTP Method
- URL
- HTTP Version

Example

```
GET /index.html HTTP/1.1
```

```
Host: www.example.com
```

```
User-Agent: Chrome
```

```
Accept: text/html
```

(No body for GET)

```
GET /students HTTP/1.1
```

Meaning

- GET → retrieve data
- /students → requested resource
- HTTP/1.1 → protocol version

2. Headers

Provide additional information.

Example

```
Host: www.example.com
```

```
User-Agent: Chrome
```

```
Accept: text/html
```

Accept

Meaning

```
Accept: application/json
```

Host

Client expects JSON.

```
Host: www.example.com
```

Which server to contact.

User-Agent

```
User-Agent: Chrome
```

Which browser is sending request.

3. Blank Line

Separates headers from body.

4. Request Body

Present mainly in

- POST
- PUT
- PATCH

Example

```
POST /student HTTP/1.1
```

```
Name=Ram
```

```
Age=20
```

Complete HTTP Request Example

```
POST /student HTTP/1.1
Host: www.college.com
User-Agent: Chrome
Content-Type: application/json
Content-Length: 32
```

```
{
  "Name": "Ram",
  "Age": 20
}
```

Explanation

POST

Create student.

Host

Server address.

Content-Type

Sending JSON.

Content-Length

Size of body.

Body

```
{
  "Name": "Ram",
  "Age": 20
}
```

Actual data sent to server.

HTTP Response Message Format

After receiving a request, the server sends a response.

Format

Structure of Response

Status Line

Headers

Blank Line

Body

1. Status Line

Contains

- HTTP Version
- Status Code
- Status Message

Example

```
HTTP/1.1 200 OK
```

Meaning

```
HTTP/1.1
```

Protocol version

```
200
```

Status code

```
OK
```

Status message

2. Response Headers

Example

```
Content-Type: text/html
Content-Length: 1500
Date: Wed, 16 July 2026
```

3. Blank Line

Separates headers and body.

4. Response Body

Contains actual content.

Example

```
<html>
<body>
Welcome to KSC
</body>
</html>
```

Complete HTTP Response Example

```
HTTP/1.1 200 OK
```

```
Content-Type: text/html
```

```
Content-Length: 45
```

```
<html>
```

```
<body>
```

```
Hello Student
```

```
</body>
```

```
</html>
```

Common HTTP Status Codes

Code	Meaning	Example
200 OK	Request successful	Page loaded successfully
201 Created	Resource created	New student added
204 No Content	Success, but no data returned	Delete operation completed
301 Moved Permanently	Resource moved	Website redirects to a new URL
400 Bad Request	Invalid request	Missing required fields in a form
401 Unauthorized	Login required	Accessing a protected page without logging in
403 Forbidden	Access denied	Student tries to access the admin page
404 Not Found	Resource not found	URL does not exist
500 Internal Server Error	Server error	Application crashes due to an exception

Suppose an ASP.NET Core application has a route:

GET /students

Client Request

GET /students HTTP/1.1
Host: localhost:5000
Accept: application/json

Server Response

HTTP/1.1 200 OK
Content-Type: application/json

```
[
  {
    "Id":1,
    "Name":"Ram"
  },
  {
    "Id":2,
    "Name":"Sita"
  }
]
```

Request-Response Example (ASP.NET Core)

The browser (or client) sends a GET request to /students, and the ASP.NET Core server processes it and returns a 200 OK response with student data in JSON format.

HTTP Request vs HTTP Response

HTTP Request

Sent by client (browser/app)

Asks for a resource or performs an action

Contains method, URL, headers, and optional body

Example: GET /students

HTTP Response

Sent by server

Returns the requested resource or result

Contains status line, headers, and optional body

Example: HTTP/1.1 200 OK

Common Web Application Architectures

Feature	Two-Tier	Three-Tier	N-Tier
Layers	2	3	Many
Complexity	Low	Medium	High
Performance	Good	Good	Good
Security	Low	High	Very High
Scalability	Poor	Good	Excellent
Best For	Small apps	Medium applications	Large enterprise systems

MVC Pattern (Model-View-Controller)

MVC (Model-View-Controller) is a software design pattern used to separate an application into three independent components. It improves code organization, maintainability, reusability, and testing.

1. Model

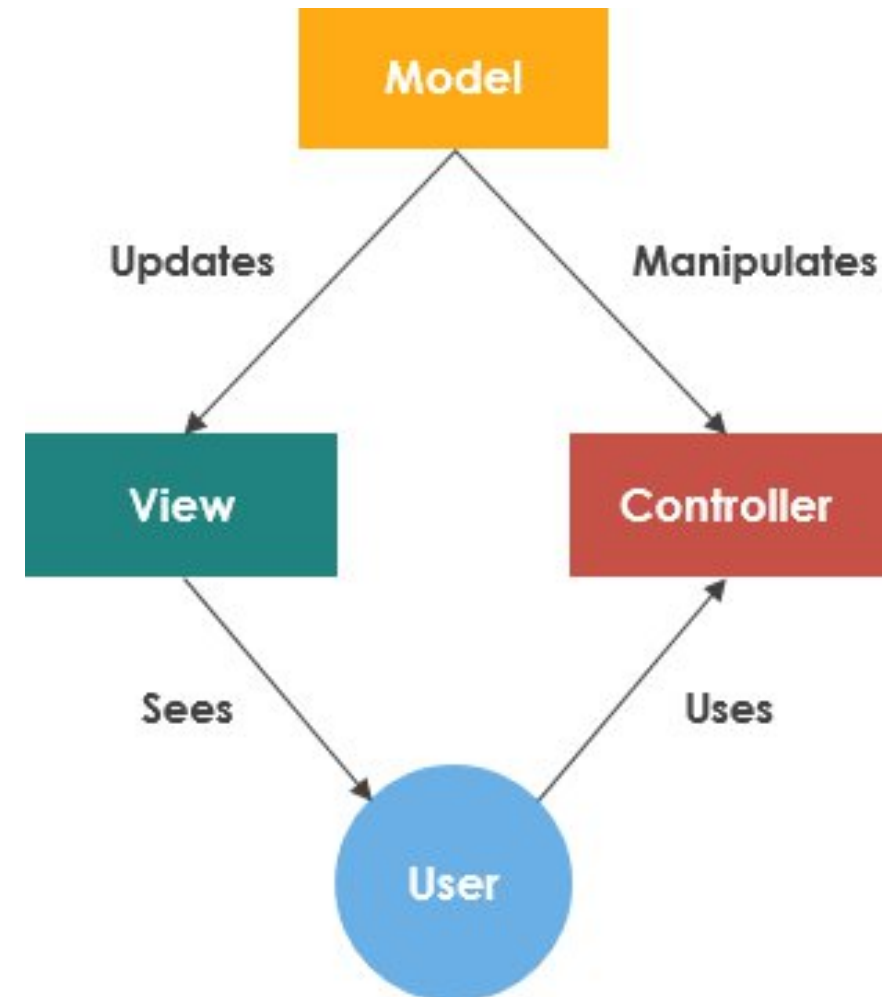
The **Model** represents the application's **data and business logic**.

Responsibilities

- Stores data
- Retrieves data from database
- Performs business rules
- Updates data

```
public class Student
{
    public int Id { get; set; }
    public string Name { get; set; }
}
```

The `Student` class is the **Model**.



2. View

The **View** is the **User Interface (UI)**.

Responsibilities

- Displays information
- Receives user input
- Contains HTML, CSS, Razor pages

Example

 HTML

```
<h2>Student List</h2>
```

```
<p>Ram</p>
```

```
<p>Sita</p>
```

This page only displays data.

3. Controller

The **Controller** acts as the **middleman** between the Model and the View.

Responsibilities

- Receives user request
- Calls Model
- Sends data to View

Example

 C#

```
public class StudentController : Controller
{
    public IActionResult Index()
    {
        return View();
    }
}
```

MVC Working Process

Suppose the user visits:

`https://college.com/students`

Step 1

Browser sends request

`GET /students`

↓

Step 2

Controller receives request.

`StudentController`

↓

Step 3

Controller asks Model for data.

`Student Model`

↓

Step 4

Model retrieves data from database.

Ram
Sita
Hari

↓

Step 5

Controller sends data to View.

↓

Step 6

View generates HTML.

↓

Step 7

Browser displays webpage.

Note: Create Project with ASP.NET Core Web App (Model-View-Controller) in Visual Studio; Do not choose "ASP.NET Core Web App (Razor Pages)". Choose the Model-View-Controller (MVC) template.

Suppose a student list application.

Model

```
</> C#  
  
public class Student  
{  
    public string Name { get; set; }  
}
```

View (Index.cshtml)

```
HTML  
  
@model List<Student>  
  
<h2>Students</h2>  
  
@foreach (var s in Model)  
{  
    <p>@s.Name</p>  
}
```

Controller

```
</> C#  
  
public class StudentController : Controller  
{  
    public IActionResult Index()  
    {  
        List<Student> students = new List<Student>()  
        {  
            new Student(){Name="Ram"},  
            new Student(){Name="Sita"}  
        };  
  
        return View(students);  
    }  
}
```

Output

Students

Ram

Sita

Advantages of MVC

- Separates application into **Model, View, and Controller**.
- Easier to maintain and modify.
- Supports code reuse.
- Easy unit testing.
- Multiple developers can work independently on different components.
- Well suited for large web applications.

Disadvantages of MVC

- More files and folders than simple web applications.
- Slightly difficult for beginners.
- May be unnecessary for very small projects.

MVC vs Traditional Web Application

Traditional Web Application	MVC Pattern
UI and business logic mixed together	Clear separation of concerns
Difficult to maintain	Easy to maintain
Less reusable code	Highly reusable
Difficult to test	Easy to unit test
Suitable for small applications	Suitable for medium and large applications

- *Describe the procedure of deploying .NET core application.*
- *Differentiate between .NET, .NET Core, and Mono frameworks. Explain the compilation and execution process of .NET applications covering CLI, MSIL, and CLR.*
- *Describe the ASP.NET Core architecture overview. How does it differ from traditional ASP.NET framework architecture?*
- *Describe the importance of MVC pattern in designing web applications.*
- *Differentiate between .NET and ASP.NET frameworks. How do you test the .NET core applications?*
- *Define HTTP. Explain the MVC pattern.*
- *Explain about request and response message format with example.*
- *Explain the process to deploy the core application.*